



A high-capacity coverless image steganography approach using block-level hash encoding and sequence category mapping: WYSAWIS

Damaris Belle M. Fotso¹ · Serge Alain Ebele^{1,2,3} · Stéphane Gael R. Ekodeck^{1,2,3} · René Ndoundam^{1,2,3}

Received: 19 September 2024 / Revised: 14 August 2025 / Accepted: 25 August 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

Existing approaches that map multiple hash sequences to a single cover element typically use an entire image as the cover. As a result, each image can represent only one hash sequence at a time, leading to the need for transmitting a large number of cover images during the hiding process. Although an image can technically represent multiple hash sequences, its effective capacity is limited to the length L of a single sequence. Conversely, methods that use image blocks as cover elements rely on one-to-one mapping between sequences and blocks, causing the number of required blocks to grow exponentially with the sequence length. Moreover, they lack a secure and well-defined method for transmitting the location information required for message extraction. This paper proposes a novel approach that combines block-level hash encoding with sequence category mapping. By categorizing sequences and mapping a sequence category to a single image block, the method increases the effective capacity of an image, allowing a single image to convey a full message. Additionally, the number of required cover blocks reduces from 2^L to $L + 1$. The secure and practical transmission of location information is ensured using a distributed data hiding technique in a single cloud environment. Our method, dubbed WYSAWIS (What You See And What It iS), highlights how seemingly innocuous image and file transfers may hide data. Compared to other coverless image steganography approaches, WYSAWIS allows grouping up to 6,435 sequences under one cover block, which reduces the number of cover blocks from 32,768 to just 16.

Keywords Coverless image steganography · Hiding capacity · Block level hash encoding · Sequence category mapping · Distributed data hiding

1 Introduction

Data security is an increasingly important issue today [1]. As new data security techniques are proposed by experts, new threats are developed by attackers. Data hiding, and steganography in particular, is one of the preferred methods to deal with these threats to the security of confidential data [2]. Its security is based on the fact that the attacker is unaware of the existence of a secret message [3]. The message is embedded in a medium file that can be a text [4, 5], audio [6, 7], video [8, 9], image [10, 11]. Due to the amount of redundant information in image files, they are one of the most commonly used media [12]. The principal core criteria of image steganography are (1) the detectability of the message (involves improving the quality of the stego image to prevent detection by human eye or steganalysis tools), (2) the payload capacity of the cover image (the amount of information that can be hidden), (3) the robustness of the stego image (enabling message extraction after distortion attacks) [13]. Traditional image steganography involves inserting the message into image pixels in the spatial domain [14, 15] and into pixels transformed into coefficients in the frequency domain [16, 17]. This modifies the stego-image, making it sensitive to image steganalysis tools [18].

To address this problem, coverless image steganography (CIS) techniques have been developed since 2015, making it possible to send a message using an image without modifying it [1]. Stego images are thus insensitive to image steganalysis tools that rely on the search for a difference between the stego image and the cover image. The methods of this new concept can be divided into two main categories [19]:

- Methods by generating the cover image [20, 21], which consists in creating the host medium from a known distribution between the two parties (e.g., embedding a secret message by creating stego texture images).
- Methods by selecting the cover image, [18, 22] which consists in extracting from a fixed image database, the image whose fingerprint (called hash sequence because it is obtained from a hash algorithm) coincides with the desired message.

Due to the high likelihood of attacks in public channels, several image selection-based CIS approaches ([23, 24]) have focused on improving robustness. However, increasing the hiding capacity remains a big challenge. Since secret information is mapped to hash sequences, the hiding capacity of an image depends both on the length of the generated hash sequences and on the number of different sequences it can represent. Moreover, as the desired capacity increases, so does the need for efficient hash sequence mapping. For a hash sequence of length L , the total number of possible sequences is 2^L . For example, with $L = 15$, there are $2^{15} = 32,768$ possible hash sequences.

Approaches designed to increase hiding capacity generally fall into two categories:

- Full-image mapping approaches: These typically extend the length of hash sequences [25, 26] and map multiple sequences to the same image [27, 28], which speeds up the mapping process. Nonetheless, the effective capacity of each image remains limited to the length of one hash sequence. Consequently, for large messages, the primary target of high-capacity CIS approaches, the number of required images becomes impractically high.

- Block-based mapping approaches [22, 29]: These increase the effective capacity of an image by leveraging multiple blocks within a single image. However, if each block is mapped to only one unique hash sequence, then 32,768 distinct blocks would still be required to cover all possible sequences of length 15. Although a single image may contain millions of blocks, the hash sequences generated from these blocks are not all unique. As a result, constructing a dataset of cover blocks that map to all distinct hash sequences is challenging.

To summarize, neither of the above categories offers a good balance between increasing hiding capacity and achieving efficient hash sequence mapping. Furthermore, the transmission of additional data, such as the number of 0 bits added, the number of target blocks, and their order, all of which are essential for message extraction, is often insecure or poorly defined. This significantly limits the applicability of these approaches in the real world.

The objective of this paper is to address the above issues by proposing a new high-capacity CIS approach that increases the hiding capacity while easily generating all possible hash sequences and sending location information in a secure and practical way.

The proposed method is built on two key contributions: (1) To achieve high capacity while ensuring the easy generation of all possible hash sequences, our mapping mechanism first groups sequences by the number of 0 bits they contain. All sequences within the same group, are then mapped to a single image block, reducing the required number of cover blocks from 2^L to $L+1$. (2) We introduce a general and practical method for transmitting location information through a distributed data hiding scheme. To the best of our knowledge, this represents the first end-to-end approach that offers a secure and fully detailed solution for conveying location information within the context of coverless steganography.

The rest of this paper is organized as follows: Section 2 presents the related work, preliminaries are given in Section 3, and the proposed coverless image steganography approach is given in Section 4. Experimental results are given in Section 5, followed with the discussions on security analysis and applications scenarios in Section 6; and Section 7 presents the conclusions.

2 Related work

The first coverless image steganography method based on mapping rules was proposed in 2015 by Zhou et al. [18]. It divides the image into 9 non-overlapping blocks, calculates the intensity of each block, and compares these 9 intensities in zigzag order, resulting in a very low hidden capacity of 8 bits. Zhou et al. proposed a better capacity of 16 bits in 2017 with a method based on histograms of oriented gradients (HOGs) [25]. It divides the image into 4×4 blocks, calculates the HOGs of each block, and then compares the HOG of each block to the average of all HOGs. If the HOG of the block is higher than the average, it is assigned a value of 1; otherwise, it is assigned a value of 0. In the same year, Zheng et al. [27] increased the capacity to 18 bits. The image is subdivided into 9 blocks and a pair of bits is mapped to each block. The set of possible arrangements of the bit pairs in these 9 blocks constitutes the set of hash sequences mapped to this image. In this case, the location information corresponding to the image block arrangements used to generate the target hash sequences must be transmitted to the receiver. Zheng et al. hide this information in a random

image using the LSB Replacement method which affects the security of the scheme due to the sensitivity of LSB Replacement to image steganalysis.

To improve the capacity while ensuring security against steganalysis, in 2018, Yi Cao et al. [30] proposed a 36-bit capacity method based on the molecular structure images of materials. It uses the average value of a sub-image's pixels to represent the secret information, according to the mapping between pixel value intervals and secret information. The receiver's identity information represents the secret key, used by the two parties to determine the number of sub-images or to pseudo-randomize the hiding process. However, no information about when and how this key is shared is defined.

For a better robustness, Zhili Zhou et al. [28] use Faster-RCNN to detect and locate objects in images and use the labels of these objects to express secret information. In 2021, Abdulsattar proposed a method with such a large capacity that one image can be sufficient to convey a complete secret message [22]. In this case, it is necessary to indicate to the receiver which blocks of the image carry the secret message. This location information is encrypted using the asymmetric PAILLIER algorithm and used between the two parties as a secret key.

Recently, deep learning approaches have been introduced in the context of coverless steganography to achieve better performance. Due to the lack of approaches in the literature dedicated to the construction of a cover image dataset, Zou et al. [26] propose a method for this purpose. The cover image dataset is built from a smaller initial image dataset, by extracting 4096-bit deep hash features from images using a CNN. These features are clustered using K-Means into 2^L groups, where L is the length of a hash sequence. One representative image from each cluster (closest to the center) is selected to form the CID. Since the process is deterministic, both sender and receiver can construct the same CID independently. However, the effective capacity remains low. Anggriani et al. [31] introduce two CIS methods that improve embedding capacity and robustness. The first E-CIHMSB (Extended Coverless Information Hiding Based on MSB) extends CIHMSB by using additional average values from image fragments and gets a hiding capacity of 20,480 bits. The second, CB-CIHMSB (Combination-Based CIHMSB) with a better capacity of 61440 bits, uses combination theory to generate more fragment combinations from the image. The limitation is linked to the one-to-one mapping rule, which maps one average value to each unique secret bit segment (e.g., 8 bits). Since there is no prior work to ensure that the image contains all required averages, the mapping may fail. To enhance the capacity while ensuring better mapping of the secret message segments, Mangi et al. [29] propose a CIS method, based on image blocks features. An image is selected and divided into 8×8 non-overlapping blocks then, hash sequence is created from the coefficient of the image block by implementing the DWT on each block to generate four subbands (low low, low high, high low, and high high). Though the length of the resulting hash sequence is 56 bits, the message is divided in 4-bits segments, which reduces the number of required hash sequences. However, due to the very small size of message segments, the location information increase significantly with the size of the message.

Another deep learning-based CIS method is presented by Li et al. [11]. The key components are: an image generation module using a disentangled autoencoder and StyleGAN to synthesize images that naturally integrate structural features from reference images and a mapping module which employs an optimized Optical Mark Recognition (OMR) mapping algorithm to embed texture features into a "bubble table," facilitating low-distortion and high-capacity steganography.

3 Preliminaries

In this section, we present the different concepts that are used in order to build our method.

3.1 Distributed data hiding method in a single cloud

In our approach, we have adapted the method proposed by Mossebo et al. (2021) [32] to securely transmit location information. Prior to communication, the sender and receiver agree upon and share the credentials of a cloud account, as well as the base used for encoding—both of which constitute the secret key (Table 1). The cloud storage environment consists of a set of cover folders containing several multimedia files. The number of cover folders should be greater than or equal to the size of the secret. The number of files in each folder should be greater or equal to the value of the base. The following relationships are checked in the cloud storage environment:

- The cover folders are all different;
- The files in the same folder are all different;
- All files in a cover folder are different from files in other cover folders.

Given a secret message s and the base B , the embedding process of s is defined as follows:

1. Create an empty folder F' that represents the stego folder;
2. Convert the secret message s in base B such that $s = \{(s_{n-1}s_{n-2} \cdots s_0)\}_B$, where $0 \leq s_i < B$;
 - 3. For each position i of the secret message s : $i = 0, 1, \dots, n - 1$ Find the file with index s_i in the folder $F^{(i)}, F_{s(i)}^i$;
 - Select and copy the file $F_{s(i)}^i$ and paste in the stego folder F' ;

3.2 Symmetric chain and shift encryption algorithms

To provide another layer of security, location information is encrypted before being sent. It was necessary to obtain ciphers that are not too large to avoid the constraint of requiring a very large cloud storage environment.

The shift encryption algorithm is used to encrypt the ranks of target block numbers. The formal definition of the shift encryption algorithm is given by [33]:

$$\begin{cases} e_k(x) = x + K \mod m \\ d_k(y) = y - K \mod m \end{cases}, \quad \text{in } \mathbb{Z}_m \quad (1)$$

where $e_k(x)$ and $d_k(y)$ represent the encryption and decryption functions respectively, K is the key, and $0 \leq K \leq m - 1$.

We use the chain encryption algorithm to encrypt target sequence numbers. Chain encryption consists in generating a key sequence $k = k_1, k_2, \dots$, and using it to encrypt the chain $x = x_1x_2 \cdots$ according to the rule: $y = y_1y_2 \cdots = e_{k_1}e_{k_2} \cdots$ [33]

Table 1 Literature review

Approach	Cover element	Capacity	Side information	Pros	Cons
Zhou et al. (2015) [18]	Image	8	-	Good robustness	Very low capacity
Zhou et al. (2017) [25]	Image	16	-	Good robustness	Very low capacity
Zheng et al. (2017) [27]	Image	18	Unsecure	Good mapping process	Low capacity
Yi Cao et al. (2018) [30]	Image	36	Unexplicit	Good mapping process	Low capacity
Zhou et al. (2019) [28]	Image	20–25	Unexplicit	Good mapping process	Low capacity
Abdulsattar (2021) [22]	Image block	55,112	Secure	High capacity	Low mapping process
Zou et al. (2022) [26]	Image	16	-	Good mapping process	Low capacity
Anggriani et al. (2023) [31]	Image block	20,480/61,440	-	High capacity	Low mapping process
Mangi et al. (2023) [29]	Image block	229,376	Secure	High capacity	Heavy side information
Li et al. (2024) [11]	Image	196,608	-	Good mapping process	Complex scheme

4 The proposed coverless steganography framework

Our goal is to increase capacity while easily generating all possible hash sequences and sending location information in a secure and practical way. We introduce the concept of sequence category mapping to a cover block, which makes it possible to map different hash sequences to a cover block.

Our hash sequence generation maps a set of hash sequences of the same category to an image block. To minimize the number of required blocks, we calculate the set of permutations for each bit in the initial hash sequence of the block. We build an inverted index structure that stores each block with the list of its hash sequences. To speed up the search for the position of a 15-bit hash sequence in the structure, we first search for the block to which it belongs, then for its rank among the sequences in that block. This position represents the location information to be transmitted.

To make the transmission less detectable, the location information is sent one by one. In general, location information has a hierarchical structure. For example, in Zheng et al.'s method, it consists of the order of the image blocks. In Abdulsattar's method it consists of the block numbers of an image, and in our method, it consists of the sequence numbers of an image block. Thus, the hierarchical hiding structure (*folder* $\rightarrow$ *file*) of the distributed data hiding approach in a single cloud of Mossebo et al. [32] motivated our choice to adapt this method for sending location information in our approach.

To send a secret message, it is first divided into 15-bit segments, with additional zeros added at the end of the last segment if the size is not a multiple of 15. The position of each of these segments is searched in the index structure, generating a pair (*block number*, *sequence number*). These pairs represent the location information, which is encrypted using chain and shift encryption algorithms, and sent using the adapted distributed data hiding technique to

a single cloud of Mossebo et al.[32]. To extract the message, the receiver uses its cloud credentials to extract the location information from the cloud and the stego image to compute the corresponding sequences that are concatenated to retrieve the secret message.

The proposed approach is detailed in five main parts, including a robust hash algorithm, inverted index structure, location information sending, secret message sending, and secret message extraction. Figure 1 presents the entities involved in the steganography process with their functions, and Fig. 2 illustrates the flowchart of the proposed coverless image steganography approach.

Figure 1 shows the following elements:

- **Sender:** responsible for sending the message. He defines the secret message and extracts the cover image from the dataset that will be used in the hiding process
- **Cover-image:** the image with which the correspondence with the secret will be established.
- **Target positions:** are the set of parts of the image containing the secret information.
- **Cloud storage environment:** Environment in which target positions or location information will be hidden.
- **Stego image:** it is the same as the cover image as it doesn't undergo changes.
- **Receiver:** responsible for extracting the message. After receiving the stego image from the sender, he checks for its ranks in the shared dataset. He uses the target positions extracted from the cloud storage along with the stego image to extract the secret that was sent.

4.1 Hash sequence generation

As the name implies, this step hashes an input image or image block, and generates an output hash sequence that represents a portion or all of the secret message. It must be as robust as possible to ensure that the hash sequences generated change as little as possible during communication.

In our algorithm, we generate from one image all the possible hash sequences necessary to communicate a message.

Hash sequences are 15-bit binary strings obtained from a robust hash algorithm described as follows:

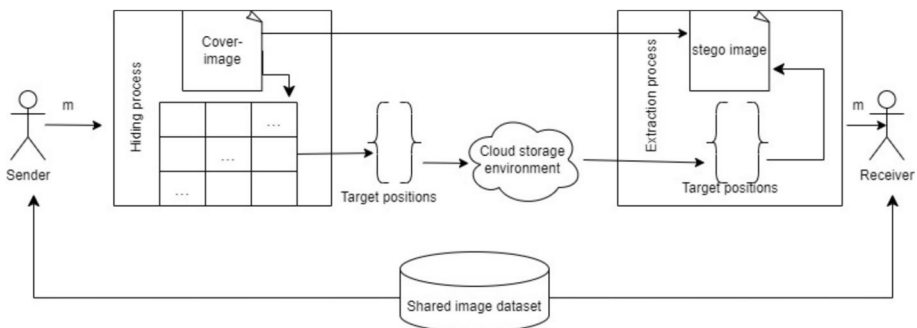


Fig. 1 Entity diagram of the proposed approach

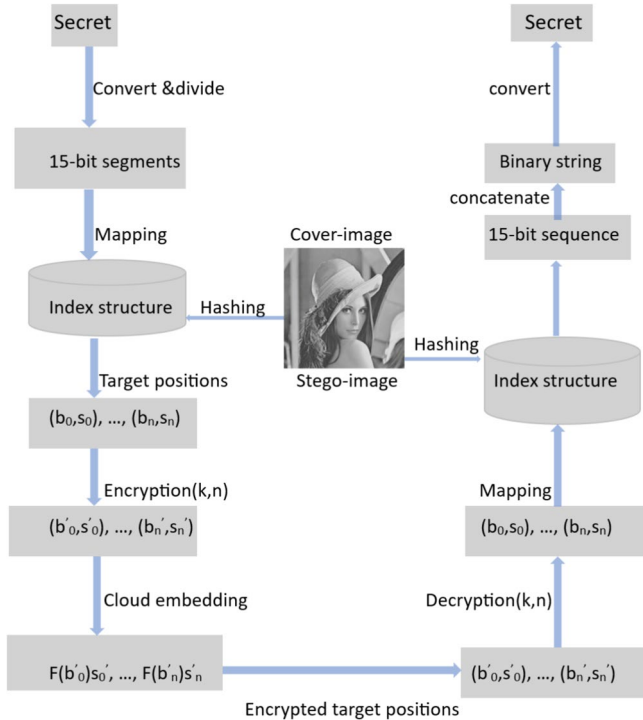


Fig. 2 The flow chart of the proposed steganography framework for secret data communication

1. The image is first converted to grayscale and normalized to $M \times N$.
2. We then subdivide the images in 4×4 non-overlapping blocks noted $\{b_0, b_1, \dots, b_j, \dots, b_{\frac{M \times N}{4 \times 4} - 1}\}$.
3. In each image block, each pixel p_i is compared to its adjacent p_{i+1} according to the equation:

$$h_i = \begin{cases} 1 & \text{if } p_i \geq p_{i+1} \\ 0 & \text{otherwise} \end{cases}, \quad \text{where } 1 \leq i \leq 15 \quad (2)$$

in the order shown in Fig. 3.

4. We obtain a 15-bit sequence with n bits at '0'. The set of possible permutations of '0' bit in the 15-bit sequence constitutes the hash sequences set associated with the block.

There are 16 categories of 15-bit binary sequence: from the one with no bits at '0' to those with 15 bits at '0'. For a sequence of n bits at '0', the set of possible permutations is :

$$P_n = \frac{15!}{n! * (15 - n)!}. \quad (3)$$

The capacity of a block corresponds to the cardinal of the set of permutations of the category of sequences in that block. Image blocks used in the continuation of the process called

useful blocks are the first to have generated a sequence category not yet indexed by the preceding blocks. Thus, there are 16 useful blocks, each of which is associated with the list of permutations of the corresponding sequence category. We called the image that contains all the 16 useful blocks *adequate*. Figure 3 shows these steps.

4.2 Inverted index structure

An inverted index is an index data structure storing the data mapping. It is this structure that establishes the correspondence between message segments and the image blocks that represent them. It will therefore store the association between all possible hash sequences and image blocks.

We numbered the image blocks according to a raster order $0, \dots, (\frac{M*N}{4*4})-1$. Sequences associated with a block are arranged in ascending order of their decimal value. To find the position of a 15-bit message segment in the structure, it would be expensive to compare it directly to the set of sequences associated with each block. So, the search is carried out as shown in Fig. 4.

1. First, we look for the sequence category of the segment. Depending on the category, we find the corresponding image block.
2. Once the block is found, its number n_b is extracted, and the segment is compared to each of the sequences in the block. When there is a match, the sequence number n_s in the block is saved, and the pair (n_b, n_s) is returned as the segment position.

4.3 Location information sending

Some changes have been made to the initial scheme described in Section 3.1 to adapt it to the transmission of location information. The secret key consists of the cloud account access and the block encryption key. The cloud storage environment consists of a set of cover folders containing several multimedia files such as:

- The number of cover folders should be greater than or equal to the number of useful blocks
- The number of files in each folder should be greater or equal to the maximum capacity of blocks.

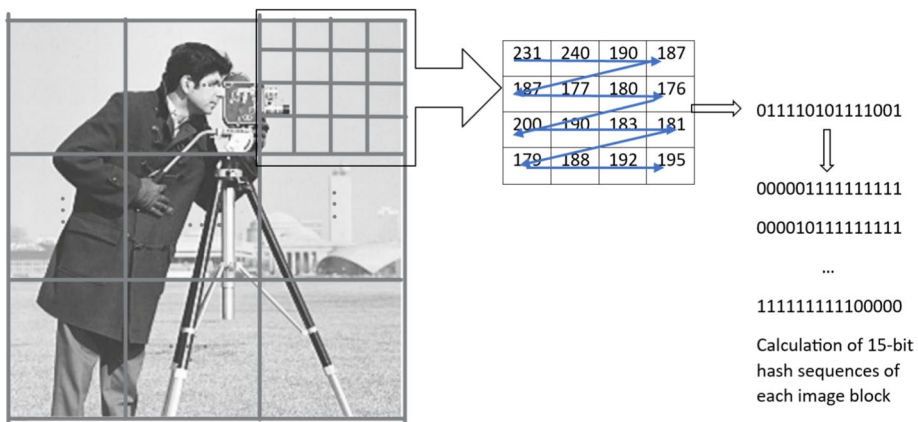


Fig. 3 The procedure of hash sequence generation by our robust hashing algorithm

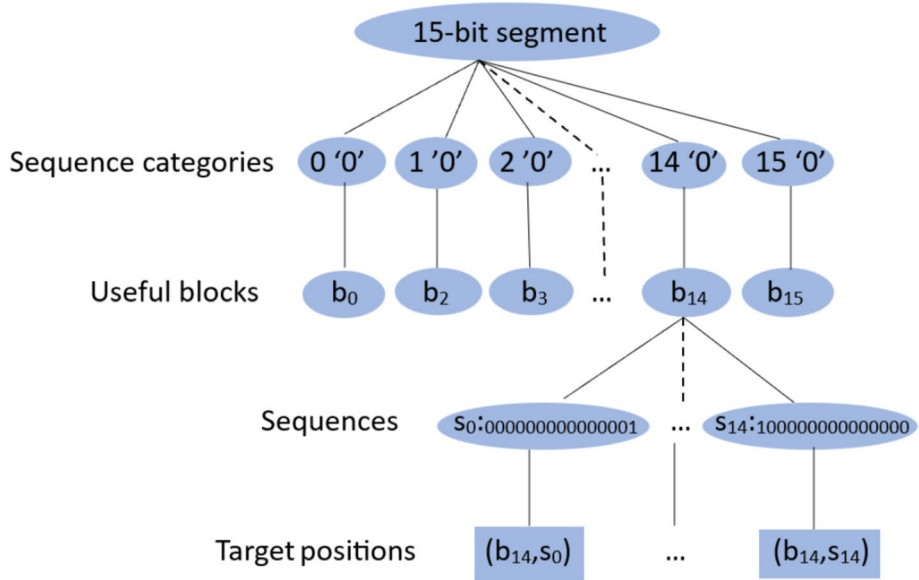


Fig. 4 The inverted index structure of the proposed approach

The location information consists of a list of pairs (block number, sequence number). Each pair is encrypted and hidden in the cloud as follows:

4.3.1 Generation of the encryption keys

The image dataset previously shared between the two parties makes it possible to have new encryption keys each time they communicate, without having to exchange them explicitly. These keys (sequence key and block key) are deduced from r as shown below :

$$\begin{cases} \text{block key} = r \\ \text{sequence key} = \begin{cases} m & \text{if } m \in [6, 8] \\ m + 6 & \text{if } m < 3 \\ 6 & \text{otherwise} \end{cases} \end{cases} \quad (4)$$

where r is the rank of the cover image in the shared dataset and $m = r \bmod 10$.

The sequence key values were set between 6 and 8 to avoid an encryption value greater than the cardinality of the stego folders.

4.3.2 Encryption of location information

Sequence numbers are encrypted using chain encryption, and block numbers using shift encryption.

4.3.3 Hiding location information in the cloud

There are 16 useful blocks and the maximum capacity is 6435 (see Equation 3). Therefore, the cloud storage environment should have at least 17 folders: 16 cover folders (each con-

taining at least 6435 files, all different) and an empty stego-folder (which by simple convention will be the first).

- For each pair of encrypted positions

$$(C(b), C(s))$$

where b refers to the block, and s to the block sequence, we copy the file at the $C(s)^{th}$ position of the $C(b)^{th}$ folder into the stego-folder.

- The exact size T of the message (the message without the added zeros) is not explicitly a location information, but is also hidden in the cloud. Since the value of T can be greater to 6435 which is the maximum number of files in a folder, we set $T = 6435 * q + r$ to ensure there will always be a stego-file for T . To send T , we copy the r^{th} file of the $(q + 1)^{th}$ folder. By convention, this is the first copied file.

4.4 Process of information hiding

This process represents the sending of the message, and is performed only after the previous two steps : using an image extracted from the shared image database, all the hash sequences have been generated, and the index structure has been built. the sender who executes it must take the following steps. Each step is followed by the name of the corresponding function:

1. If the length of the message converted to binary is not a multiple of 15, zeros are added at the end: (`convertToBinary()`).
2. The sender extracts a random image from the image dataset and its rank r in this dataset. It must be different for each new communication: (`random()`).
3. Message is divided into 15-bit segments: (`divideIntoSegments()`).
4. For each 15-bit segment of the message, we find its position from the inverted index structure of the image blocks: (`searchIndexStructure()`).
5. Location information sending: This includes the generation of the encryption keys (`generateKeys()`), encryption of location information (`encryptLocation()`) and the hiding in the cloud (`sendToCloud()`).

The pseudo-code of the information hiding process is described in algorithm 1. Each of the functions used is associated with one of the above steps.

Algorithm 1 Hiding process

Require: M : secret, dt : shared dataset, credentials: cloud credentials
Ensure: I : stego image, C : modified cloud account

- 1: $M' = \text{convertToBinary}(M)$
- 2: $\text{segments} = \text{divideIntoSegments}(M')$
- 3: $(I, r) = \text{random}(dt)$
- 4: $\text{locationPairs} = \text{searchIndexStructure}(\text{segments}, I)$
- 5: $\text{keys} = \text{generateKeys}(r)$
- 6: $\text{encryptedLocation} = \text{encryptLocation}(\text{locationPairs}, \text{keys})$
- 7: $C = \text{sendToCloud}(\text{encryptedLocation}, \text{credentials})$
- 8: **Return** $\{I, C\}$

4.5 Process of information extraction

It can be considered as the reverse function of the message hiding process, and generally concludes the secret data communication in the steganographic scheme. The extraction of the message on the receiver side proceeds as follows:

1. After receiving the image, the receiver searches for it in its image data set which is identical to that of the sender, and finds out its rank r : `rank()`.
2. Using r , the receiver computes the encryption keys: `generateKeys()`.
3. The receiver logs on the same cloud and searches for each of the files in the stego folder in the cover folders, and gets the positions (folder number, file number): `retrieveFromCloud()`.
4. These positions (except the first one because it corresponds to the size of the message) are decrypted using the calculated encryption keys and the corresponding algorithms, which give the target positions of the sequences (block number, sequence number): `decryptLocation()`. This step has the advantage of indicating to the receiver whether the image has been modified, as well as any erroneous parts of the extracted message.
5. Using the stego image, the hash sequences corresponding to the decrypted positions are calculated. All these sequences are concatenated in order: `computeAndConcatenate()`.
6. Depending on whether the size ($T = \text{folder number} * 6435 + \text{file number}$) is a multiple of 15 or not, some zeros are removed or not: `zeros()`.
7. the sequence is converted from binary to the appropriate base to have the secret message M : `convertFromBinary()`

The process of information extraction is formally described in algorithm 2. As above, each of the functions used is associated with one of the above steps.

Algorithm 2 Extraction process

Require: I : stego-image, dt : shared dataset, credentials: cloud credentials of the modified account

Ensure: M : secret

- 1: $r = \text{rank}(I, dt)$
 - 2: $\text{keys} = \text{generateKeys}(r)$
 - 3: $\text{encryptedLocation} = \text{retrieveFromCloud}(\text{credentials})$
 - 4: $\text{locationPairs} = \text{decryptLocation}(\text{encryptedLocation}, \text{keys})$
 - 5: $M' = \text{computeAndConcatenate}(\text{locationPairs}, I)$
 - 6: $M = \text{convertFromBinary}(M')$
 - 7: **Return** M
-

4.6 Time complexity

In this subsection, we evaluate the time complexity of the proposed approach. Given a secret message M :

4.6.1 Hiding process

- Conversion of M into binary and subdivision into n segments of 15 bits: $O(\log 2(M)) + c_1$, c_1 is a constant.
- Search for the position of the cover image in the image dataset: $O(m)$, where m is the size of the image dataset.
- Compute the encryption keys: $O(3)$.
- Search for target positions: each of the n segments is searched from the list of sequences in the corresponding block. This gives a complexity of $O(n * c)$ where c is the capacity of the corresponding block.
- Target position encryption: for each of the n pairs, 2 additions are made, the addition of the block number to the block key and the addition of the sequence number to the sequence key. This gives $O(2n)$.
- Searching for positions in the cloud: this involves copying the file corresponding to each of the n positions into the stego-folder. This is done in $O(n)$.

The time complexity of the hiding process is therefore $O(n * c)$

4.6.2 Extraction process

- The search for each of the n stego-files in the corresponding cover folder is done in $O(n * C)$ where C is the size of the cover folder.
- Search for the position of the stego-image in the image dataset: $O(m)$.
- Calculate the encryption keys: $O(3)$.
- Decryption of target positions: $O(2n)$.
- Search for the sequence in the image corresponding to each of the n target positions is done in $O(n)$.
- The sequences are concatenated and converted into the base B of the secret message in $O(\log B(M)) + c_2$, c_2 is a constant.

The time complexity of the extraction process is therefore $O(n * C)$.

5 Experimentation

5.1 Example

In this subsection, we describe in detail the processes of embedding and extraction of the secret of the approach presented. In this example, Alice wants to send Bob the french secret message $M = \text{"\#ceci est un message très secret. Exécutez très rapidement toutes les directives: heure d'embarquement 13h, lieu:aéroport central"}$. They each have shared access to an image dataset and to a free Google Drive account consisting of 16 cover folders each containing 6435 files, all different and distinct from the files in the other folders, and an empty stego folder. The cloud storage environment is shown in Figs. 6 and 7.

5.1.1 On the sender side

Alice extracts an image from the shared image dataset (Fig. 5) and checks whether it is an *adequate* image by generating the useful blocks.

Hash sequence generation Figure 8 shows the contents of the first 5 blocks of the image.

We deduce from Fig. 9 that block 2 will not be a useful block, as it generates a sequence category already mapped to block 0. Alice thus finds all 16 sequence categories mapped to



Fig. 5 The cover-image extracted from the shared image dataset

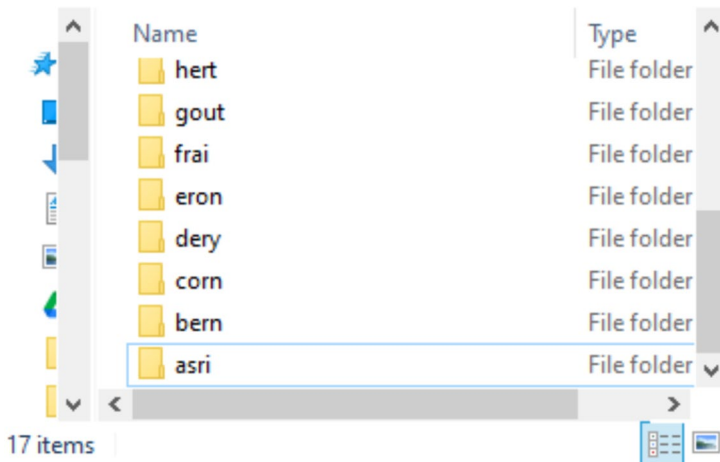


Fig. 6 Folders in cloud storage

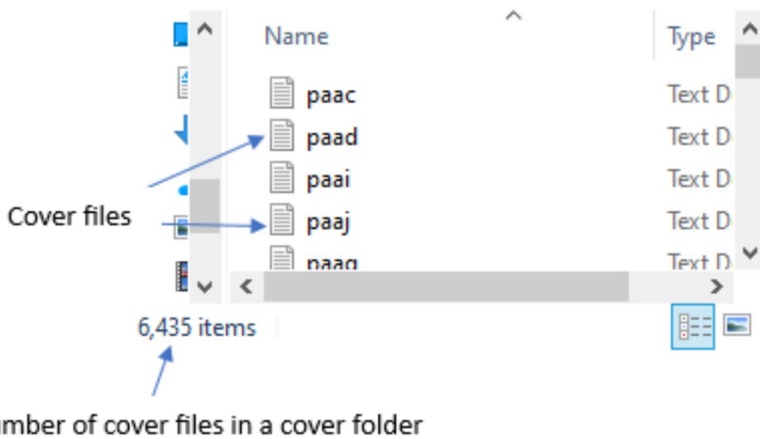


Fig. 7 Overview of a cover folder

Block 0	Block 1	Block 2	Block 3	Block 4
[[115 123 132 135]]	[[141 144 144 144]]	[[148 154 161 163]]	[[167 163 163 158]]	[[156 159 157 152]]
[[118 125 132 137]]	[[141 142 139 140]]	[[139 144 150 152]]	[[158 154 154 153]]	[[153 154 154 154]]
[[120 127 133 138]]	[[142 141 136 135]]	[[131 134 140 143]]	[[152 152 152 158]]	[[162 164 167 171]]
[[120 127 133 138]]	[[142 141 136 135]]	[[131 134 140 143]]	[[152 152 152 158]]	[[162 164 167 171]]

Fig. 8 Content of the first 5 blocks of the image

the 16 useful blocks: '0', '1', '3', '4', '5', '6', '10', '14', '21', '22', '38', '128', '130', '244', '858', '13179' and can therefore use the image as the cover image.

Construction of the inverted index structure The 16 useful blocks and the corresponding hash sequences are organised in a search structure as shown in Fig. 10. Blocks 1 and 3 are mapped to sequences of 5 bits at 0 and 2 bits at 0 respectively. From the formula 3 we find that the capacities of blocks 1 and 3 are $C_1 = 3003$ and $C_3 = 105$ respectively.

Location information sending

1. Generation of the encryption keys Alice then calculates the average m of the image and searches in the file of averages for the rank r of m . This rank designates the block key and the sequence key is deduced as in the equation 4. We have $m = 97$, $r = 22$ so the block key = 22, $22 \bmod 10 = 2$, the sequence key = $2 + 6 = 8$. Figure 11 gives an overview of the averages file.

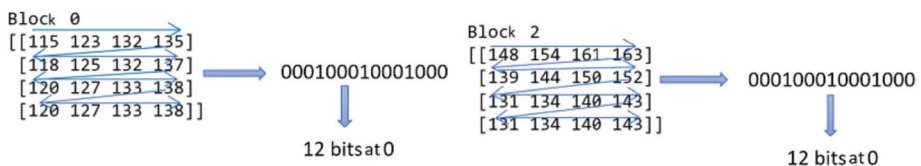


Fig. 9 Calculation of sequence categories mapped to blocks

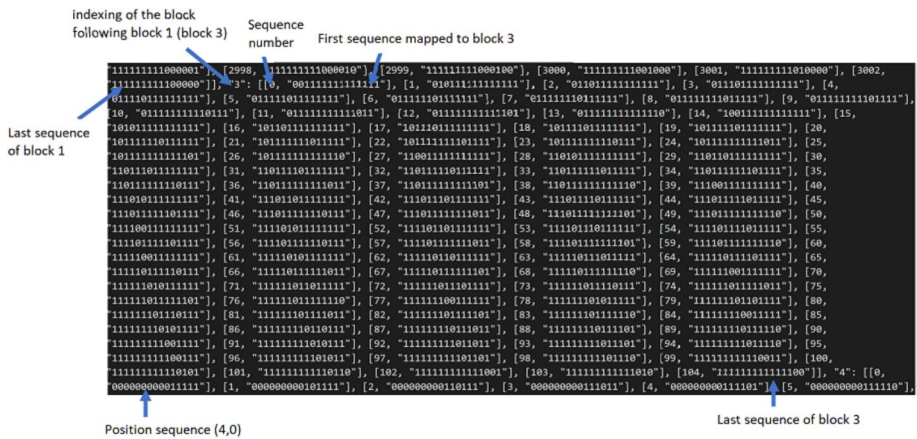


Fig. 10 Organisation of some cover blocks in the index structure



Fig. 11 Average cover images

2. Location information encryption The ranks of the blocks are thus encrypted using the shift encryption in Z_{16} (because there are 16 useful blocks). Block 0 has rank 0 (because it is the first useful block obtained), $0+22=22, 22 \bmod 16 = 6$. Block 0 will therefore be represented by the cover folder of rank 6 (7^{th} cover folder because we are counting from 0). The encryption of each block is given in Table 2:

The secret message $M = \text{"#ceci est un message très secret. Exécutez très rapidement toutes les directives: heure d'embarquement 13h, lieu:aéroport central"}$ is converted in binary, resulting in a 1032-bit binary sequence $M = \text{"0010001101100011011001010110001101101001001000000110..."}.$ This size is increased to 1035 by adding 3 '0' bits at the end to obtain a multiple of 15. Then, M is subdivided into 15-bit segments. Let S_0 be the first 15-bit segment of the message M . $S_0 = 001000110110001 \iff 10$ bits at '0'. The image block mapped to the 10-bit '0' category is the block 21 and the S_0 segment corresponds to its 1031st sequence. The sequence number 1031 will therefore be encrypted using the chain encryption with key=8, which gives $9134 \bmod 6435 = 2699$. 2699 obtained after modulo is different from 2699 obtained directly without modulo. So to avoid collisions, we perform a new hash and find the cover file at position 3809 of the cover folder 3 (see Figs. 12 and 14).

Secret information sending For each segment of the message, its position in the index structure is searched, and the corresponding cover file is copied into the stego-folder. M has $\frac{1035}{15} = 69$ segments. But the stego-folder will have 70 files. The additional file, first copied, indicates the initial size of the message, 1032 bits for our example. $1032 = 0 \times 6435 + 1032$, so it will be represented by the file at position 1032 of the first cover folder, as shown in

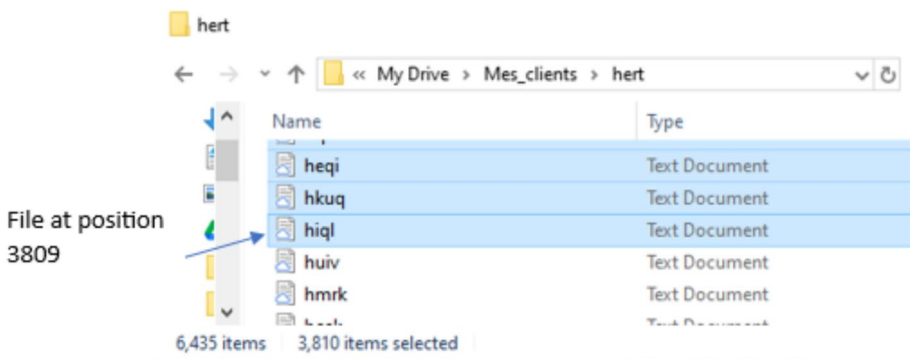
Table 2 Cover block number with the corresponding cover folder

Cover block number	Encrypted value (rank of the cover folder in the cloud)
0	6
1	7
3	8
5	10
6	11
10	12
14	13
21	14
22	15
38	0
128	1
130	2
244	3
858	4
13179	5

Figs. 13 and 14. Finally, Alice sends to Bob via any channel the stego image, identical to the cover image.

5.1.2 On the receiver side

Bob receives the stego-image and begins by generating the useful blocks. If it has not been modified during its transfer, they will be identical to those obtained by Alice. He then proceeds to extract the location information from the cloud storage environment. The files in the stego-folder are arranged in reverse order to the message segments. He searches each of the first 69 stego-files in the stego folders and gets their aaposition. The 69 15-bits segments of the message are found by searching the segments at these positions in the stego-image. Then they are arranged in reverse order and concatenated. Searching for the 70th file, he finds it at position 1032 of the 1st cover folder. $69 \times 15 = 1035$, $1035 - 1032 = 3$. He therefore removes the last 3 '0' bits from the binary sequence to obtain the message.

**Fig. 12** Position of the cover file corresponding to the first segment of the message

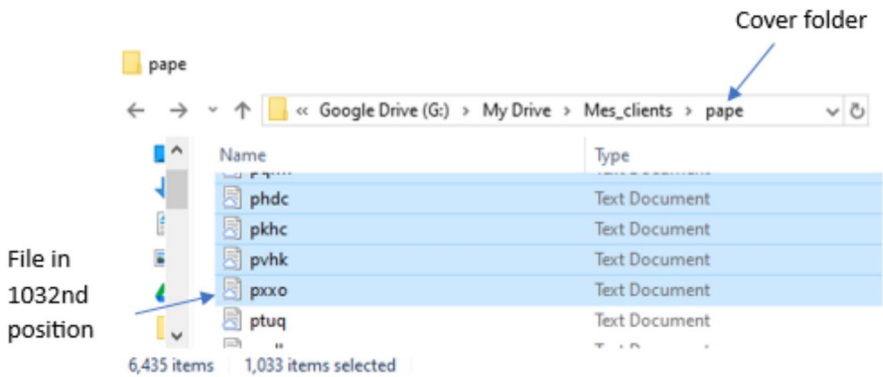


Fig. 13 Position of file representing message size

Name	Owner	Last...	▼	↓
dtqx.txt	me	09:16		
hiql.txt	me	09:15		
pxxo.txt	me	09:15		

File copied for the first segment

First file copied - file for message size

Fig. 14 Stego files in the stego folder

5.2 Experimental results and analysis

To highlight the contributions of the proposed coverless image steganography method, we performed a series of experiments to evaluate its security, robustness, and capacity, compared to other state-of-the-art methods. The experiments are conducted on three widely used image datasets: INRIA Holidays [34], which contains 1,491 high-resolution images of various natural and man-made scenes captured under diverse conditions; Caltech-256 [35], comprising 30,608 images from 257 object categories; and the Natural Images dataset [36], containing thousands of everyday object and person images. These datasets ensure diversity in content, scale, and complexity, enabling a comprehensive evaluation of the method's performance. Tests are carried out on a computer with the following specifications: Intel(R) Core™ i7-8550U CPU @ 1.9 GHz, 16 GB memory, and Windows 11 (64 bits). All codes are written in Python 3.

5.2.1 Hash sequence generation analysis

In this section, the difficulty of generating all the necessary hash sequences is evaluated. In this paper, we described the generation of 15-bit hash sequences, but it is possible to obtain

Table 3 Number of cover images required for different approaches

Approach	Hash sequence length (bits)		
	8	15	18
Initial scheme	256	32768	262144
Zheng et al. [27]	-	-	220
Abdulsattar [22]	1	-	-
Zou et al. [26]	256	32768	262144
WYSAWIS	1	1	1

The results of our proposed approach are highlighted in bold

Table 4 Number of cover-blocks required for different approaches

Approach	Hash sequence length (bits)	
	8	15
Abdulsattar [22]	256	-
WYSAWIS	9	16

The results of our proposed approach are highlighted in bold

hash sequences of greater (respectively smaller) length by increasing (respectively decreasing) the size of the image blocks. For a hash sequence of size n bits, the size of the required image database considering the traditional scheme where an image corresponds to a single hash sequence is 2^n . With WYSAWIS, the size of the required cover image database does not increase with the length of the hash sequences, but is always equal to 1. As well as using a single image, our approach reduces the number of useful blocks from 2^n to $n + 1$. Tables 3 and 4 show the efficiency of our hash sequence generation algorithm.

5.2.2 Analysis of coverless image dataset construction

Given an initial image dataset, we evaluate which approach can generate a maximum length of hash sequence, with a smaller image dataset size. As shown in Table 5, our approach generates the maximum hash sequence length.

5.2.3 Embedding capacity

Embedding capacity does not depend on the size of the image, but it would be better to use a somewhat large one (e.g. 512*512) to increase the number of blocks and thus increase the chances of getting all the useful blocks. To this end, it would also be beneficial if the image had several details. The minimum capacity is given by:

$$C = (H - 1) * \sum_{i=0}^{n-1} C(b_i) \quad (5)$$

Table 5 Hash sequence length on different image dataset size

	Holidays (1493)	Caltech-256 (903)
Zou et al.[26]	10	9
WYSAWIS	15	15

The results of our proposed approach are highlighted in bold

where H denotes the size of the image blocks, n is the number of useful blocks, and $C(b_i)$ the capacity of block b_i , since we only use an image if it is adequate, i.e. it has all the useful blocks, $n=16$ and $\sum_{i=0}^{n-1} C(b_i) = \sum_{i=0}^{n-1} P_i = 32768$ as shown in section 3. Moreover, each 15-bit segment of the message can always correspond to a sequence of image blocks. The maximum embedding capacity of an image is then theoretically infinite. In practice, this capacity is only limited by the time it takes to hide and retrieve location information from the cloud storage environment.

Table 6 shows the hiding capacity of the different approaches. It is obvious that the proposed approach provides a higher capacity than the existing methods, even if we consider the minimum capacity.

5.2.4 Robustness

It is well known that the image, during the communication process, can suffer many typical attacks that damage its content, among which we can find noise adding (speckle noise, salt and pepper noise, Gaussian noise), filtering (median filter, gaussian filter), and so on. Resilience to these image attacks is therefore an important aspect of image steganography methods, so that the secret message can be extracted with the best accuracy.

A modification of the image in our method can result in a modification of the sequence category of a useful block, and consequently its embedding capacity, from a greater to a lower one and vice-versa. In the case of a reduced capacity, segments with positions higher than the new capacity are not recovered, resulting in a few content losses. We use the BER (Bit Error Rate) and the ER (Extraction Rate) to test the robustness of the approach. The lower the BER, the better, and the higher the ER, the better. The BER is given by [22]:

$$BER = \frac{\sum_{r=1}^z xor(a_r, b_r)}{z} \quad (6)$$

where a_r denotes the hidden message, b_r the extracted message after an attack, and z the message size in bits.

The ER is given by:

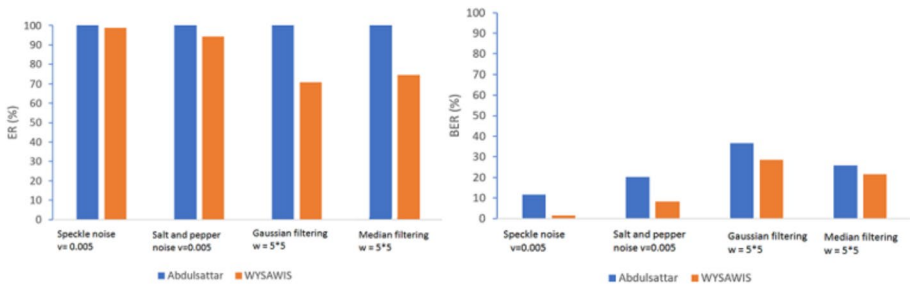
$$ER = \frac{n_e}{n_t} \quad (7)$$

Table 6 Comparison of the hiding capacity of different coverless image steganography approaches

Approach	Capacity (bits/image)
Zou et al. [26]	16
Zheng et al. [27]	18
Faster-RCNN [28]	20 and 25
MSIM [30]	36
Zou et al. [37]	16
Abdulsattar [22]	6,272 or 55,112
Meng et al. [38]	80
Anggriani [31]	20,480 or 61,440
Mangi et al. [29]	229,376
Li et al. [11]	196,608
WYSAWIS	491,520– ∞

Table 7 Resilience to image processing attacks

Attacks	Criteria	Abdulsattar	WYSAWIS
Speckle noise	BER	11.68	1.47
$v = 0.005$	ER	100	98.79
Salt and pepper	BER	20.08	8.33
noise $v = 0.005$	ER	100	94.26
Gaussian filtering	BER	36.53	28.56
$w = 5 \times 5$	ER	100	70.93
Median filtering	BER	25.90	21.57
$w = 5 \times 5$	ER	100	74.40

**Fig. 15** Resilience to image processing attacks

where n_e designates the number of extracted bits, and n_t the total number of hidden bits.

Table 7 and Fig. 15 give a comparison of the robustness of our algorithm and that of Abdulsattar considering 9×9 overlapping blocks. Results show that, even if the extraction rate is still low, the quality of the information extracted with WYSAWIS is good.

5.2.5 Location information sending analysis

The step of sending the location information is crucial in the process of communicating the secret message. In fact, without the location information, the receiver will not be able to extract the secret information, even with the image. For the sender responsible for sharing it, it must be practical without compromising the security of the general scheme. We compare our method of sending location information with two other state-of-the-art methods: Zheng's method [27] and Abdulsattar's method [22].

In Zheng's method, location information is sent using the LSB replacement. As far as we know, the LSB replacement, as one of the traditional image steganography techniques, is sensitive to image steganalysis and therefore not secure. Abdulsattar's method provides better security by encrypting the location information and sharing it as a secret key. However, knowing that secret key sharing remains a difficult task in general, and that no specific method is proposed, location information sharing remains difficult to implement. The proposed approach combines security and practicality in the transmission of location information. The expansion and popularisation of cloud services make it very easy to access a cloud, hence the practicality of our method. Moreover, the search and copy operations performed

in the process of hiding and retrieving the secret message are common operations for managing a cloud account. This gives our method an innocuous character, and thus a high level of security.

6 Discussions

An advantage of our method is that each segment of the message can always be mapped to a sequence of a useful block, while an inconvenience is that if a 15-bit segment is repeated several times in the message, the same file is copied several times in the stego folder. In Abdulsattar's method, to ensure that a block is not used several times if the ASCII code character to which it is associated is repeated in the message, a variable called the mask is defined. If the mask of a block is 0, it can be used, but if the mask is 1, it has already been used and can no longer be used. The disadvantage is that if the number of blocks associated with an ASCII code is less than the number of repetitions of that ASCII code in the message, then some characters in the message cannot be mapped to blocks. As a result, the message cannot be completely hidden using one image. In WYSAWIS, we can consider the position of the segment in the message when encrypting its position in the index structure. Thus, no matter how many times a segment is repeated in the message, it can always be mapped to a sequence of image blocks but will be represented by a different stego file, depending on its position in the message.

6.1 Security analysis

Instead of embedding the secret data directly in the image, we established the relationship between the images and the secret data using a hash map. As a result, the embedding of the secret message is undetectable to both the human eye and steganalysis algorithms because the carrier image is not modified.

Our information-hiding approach involves three main parameters: stego image, encryption keys, and cloud access. Assuming that a new image and different keys are used for each new secret message communication, we presented below, different spy attack schemes on the proposed approach based on five hypotheses.

Hypothesis 1: The attacker has the image, access to the cloud, and encryption keys: he will be able to find out the secret message.

Hypothesis 2: The attacker has the image and access to the cloud environment: he will need the key to decrypt the positions. If he tries a brute force attack, he will have to try n ways to get the block cipher key, where n is the size of the database shared by the two parties. From this, he can deduce the sequence encryption key.

Hypothesis 3: The attacker has the image without access to the cloud environment: he won't be able to extract the message because he doesn't have the location information hidden in the cloud.

Hypothesis 4: The attacker has access to the cloud storage but not to the image: if he doesn't have the key, he won't have the location information yet. If he has the key, he will have the location information, but he will not be able to match the image to find the target

sequences. A brute force attack to determine the sequence category of each cover folder will cost $16! \approx 2.09 * 10^{13}$ attempts.

Hypothesis 5: The attacker has neither access to the cloud environment nor the image: he won't be able to retrieve the message. To retrieve the message, an attacker must have a set of parameters (the stego image, the cloud access, and the cipher keys), otherwise, he would have to brute-force millions of attempts. We have thus demonstrated the high security of the proposed approach.

6.2 Application scenarios

We have identified several application scenarios that can use our technique, some of which are as follows:

- **Medical Imaging and Patient Records:** Patient data varies in size, and can be very large. This makes our high-capacity coverless image steganography method suitable for embedding patient data within medical images to provide a secure and covert method for sharing sensitive medical information.
- **Military and Defense Communications:** our approach can be used to conceal messages or intelligence data to ensure secure and discreet communication in military contexts.
- **Financial Transactions and Authentication:** Our approach, which includes hiding location information in the cloud, is appropriate for embedding cryptographic information or transaction details within images to enhance the security of financial transactions.

7 Conclusion

In this paper, we presented WYSAWIS (What You See And What It iS), a high-capacity coverless image steganography method. The approach introduces two key improvements: a well-balanced trade-off between increased hiding capacity and efficient hash sequence mapping, and a secure, practical mechanism for transmitting additional data. For length hash sequences L , the first improvement is achieved by categorizing sequences according to the number of 0 bits and mapping each sequence category to a corresponding image block. As a result, this strategy significantly reduces the number of required cover blocks from 2^L to $L + 1$. The second improvement involves securely transmitting the location information inherently produced during hashing through a distributed data-hiding technique within a single cloud environment. Experiments on standard datasets demonstrate that WYSAWIS achieves a minimum embedding capacity of 491,520 bits per image, while maintaining reasonable robustness under common image distortions. Compared to existing methods, WYSAWIS achieves higher hiding capacity, better security, and greater practicality. However, when identical message segments occur multiple times, the corresponding stego files are duplicated in cloud storage, which may arouse the suspicion of attackers. Additionally, the extraction rate tends to decrease slightly when the stego image is altered or distorted during transmission. Future work will aim to eliminate redundancy in stego-file storage and enhance the robustness of the method against image content modification.

Acknowledgements This work was supported by UMMISCO, LIRIMA, and the University of Yaounde I - Cameroon.

Author contributions Material preparation, data collection, and analysis were performed by Damaris B.M. Fotso, Serge A. Ebele, and Stéphane Gael R. Ekodeck. Conceptualization of the study was ensured by Damaris B.M. Fotso and Serge A. Ebele, and Methodology by René Ndoundam. The first draft of the manuscript was written by Damaris B.M. Fotso and all authors commented on previous versions. All authors read and approved the final manuscript.

Funding This work has no funding.

Data availability The image database that supports the findings of this study is openly available in Google Drive at <https://drive.google.com/file/d/1VTN58rQ1BLtyS4Ea7p1E1NzeHFGY2D1V/view?usp=sharing> and the Python implementation of WYSAWIS is openly available also in Google Drive at <https://drive.google.com/file/d/1hAsuNIUQ9VrrKxhvlJqV0vu-3iPsmWp/view?usp=sharing>.

Declarations

Ethical approval This study does not contain any studies with human or animal subjects performed by any of the authors.

Competing interests The authors declare they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Qin J, Luo Y, Xiang X, Tan Y, Huang H (2019) Coverless image steganography: a survey. *IEEE Access* 7:171372–171394
2. Zhou Z, Mu Y, Wu QMJ (2019) Coverless image steganography using partial-duplicate image retrieval. *Soft Comput* 23(13):4927–4938 (**Springer**)
3. Yahya A (2019) *Steganography techniques for digital images*. Springer, New York, USA
4. Guan B, Gong L, Shen Y (2022) A novel coverless text steganographic algorithm based on polynomial encryption. *Secur Commun Netw* 2022(1):1153704
5. Rashid SH, Nasrawi DA (2024) Coverless text information hiding based on built-in features of Arabic scripts. *J Appl Data Sci* 5(2):653–667
6. Li J, Wang K, Jia X (2023) A coverless audio steganography based on generative adversarial networks. *Electronics* 12(5):1253
7. Feng Y, Xu L, Lu X, Zhang G, Rao W (2025) A robust coverless audio steganography based on differential privacy clustering. *IEEE Trans Multimedia*. <https://doi.org/10.1109/TMM.2025.3543107>
8. Meng L, Jiang X, Sun T, Zhao Z, Xu Q (2023) A robust coverless video steganography based on the similarity of inter-frames. *IEEE Trans Multimed* 26:5996–6011
9. Meng L, Jiang X, Zhang Z, Li Z, Sun T (2024) A robust coverless video steganography based on maximum DC coefficients against video attacks. *Multimed Tools Appl* 83(5):13427–13461
10. Rustad S, Syukur A, Andono PN et al (2022) Inverted LSB image steganography using adaptive pattern to improve imperceptibility. *J King Saud Univ* 34(6):3559–3568
11. Li Z, Zhang Q, Li X (2024) A novel large capacity coverless image steganography method based on hybrid framework of image generation-mapping. *Multimed Syst* 30(6):1–23
12. Ekodeck SGR, Ndoundam R (2016) Pdf steganography based on Chinese Remainder Theorem. *J Inf Secur Appl* 29:1–15
13. Abdulla AA (2015) *Exploiting similarities between secret and cover images for improved embedding efficiency and security in digital steganography* (Doctoral dissertation, University of Buckingham)
14. Yang C-H, Weng C-Y, Wang S-J, Sun H-M (2008) Adaptive data hiding in edge areas of images with spatial LSB domain systems. *IEEE Trans Inf Forensics Secur* 3(3):488–497
15. Holub V, Fridrich J, Denemark T (2014) Universal distortion function for steganography in an arbitrary domain. *EURASIP J Inf Secur* 2014:1–13 (**Springer**)
16. Huang F, Huang J, Shi Y-Q (2012) New channel selection rule for JPEG steganography. *IEEE Trans Inf Forensics Secur* 7(4):1181–1191
17. Zhiwei K, Liu J, He Y (2007) Steganography based on wavelet transform and modulus function. *J Syst Eng Electron* 18(3):628–632 (**Elsevier**)

18. Zhou Z, Sun H, Harit R, Chen X, Sun X (2015) Coverless image steganography without embedding. In: Cloud computing and security: First international conference, ICCCS 2015, Nanjing, China, August 13–15, 2015. Revised Selected Papers. Springer, pp 123–132
19. Zhang X, Peng F, Long M (2018) Robust coverless image steganography based on DCT and LDA topic classification. *IEEE Trans Multimed* 20(12):3223–3238
20. Xu J, Mao X, Jin X, Jaffer A, Lu S, Li L, Toyoura M (2015) Hidden message in a deformation-based texture. *The Visual Comput* 31:1653–1669 (**Springer**)
21. Li G, Feng B, He M, Weng J, Lu W (2023) High-capacity coverless image steganographic scheme based on image synthesis. *Signal Process: Image Commun* 111:116894 (**Elsevier**)
22. Abdulsattar FS (2021) Towards a high capacity coverless information hiding approach. *Multimed Tools Appl* 80(12):18821–18837 (**Springer**)
23. Liu X, Li Z, Ma J, Zhang W, Zhang J, Ding Y (2022) Robust coverless steganography using limited mapping images. *J King Saud Univ-Comput Inf Sci* 34(7):4472–4482
24. Tan Y, Xiang X, Qin J, Tan Y (2024) Robust coverless image steganography based on human pose estimation. *Knowledge-Based Systems* 296:111873
25. Zhou Z, Wu QMJ, Yang C-N, Sun X, Pan Z (2017) Coverless image steganography using histograms of oriented gradients-based hashing algorithm. *J Internet Technol* 18(5):1177–1184
26. Zou L, Li J, Wan W, Wu QMJ, Sun J (2022) Robust coverless image steganography based on neglected coverless image dataset construction. *IEEE Trans Multimed* 25:5552–5564
27. Zheng S, Wang L, Ling B, Hu D (2017) Coverless information hiding based on robust image hashing. In: Intelligent computing methodologies: 13th international conference, ICIC 2017, Liverpool, UK, August 7–10, 2017, Proceedings, Part III. Springer, pp 536–547
28. Zhou Z, Cao Y, Wang M, Fan E, Wu QMJ (2019) Faster-Rcnn based robust coverless information hiding system in cloud environment. *IEEE Access* 7:179891–179897
29. Mangi HT, Ali SA, Jawad MJ (2023) Enhancing of coverless image steganography capacity based on image block features. *TELKOMNIKA (Telecommun Comput Electron Control)* 21(6):1364–1372
30. Cao Y, Zhou Z, Sun X, Wang M, Wu QMJ (2018) Coverless information hiding based on the molecular structure images of material. *J Inf Hiding Multimed Signal Process* 9:1–10
31. Anggriani K, Chiou S-F, Wu N-I, Hwang M-S (2023) A robust and high-capacity coverless information hiding based on combination theory. *Informatica* 34(3):449–464
32. Mossebo Tcheunteu SW, Moyou Metcheke L, Ndoundam R (2021) Distributed data hiding in a single cloud storage environment. *J Cloud Comput* 10(1):43
33. Stinson D, Vaudenay S, Avoine G, Junod P (2003) *Cryptographie – Théorie et pratique* (2ème édition). Vuibert, Paris, France
34. Jegou H, Douze M, Schmid C (2008) Hamming embedding and weak geometric consistency for large scale image search. In: Computer Vision–ECCV 2008: 10th European conference on computer vision, Marseille, France, October 12–18, 2008, Proceedings, Part I. Springer, pp 304–317
35. Holub AD, Perona P, et al. (2006) The Caltech-256. Caltech Technical Report
36. Roy P, Ghosh S, Bhattacharya S, Pal U (2018) Effects of degradations on deep neural network architectures. [arXiv:1807.10108](https://arxiv.org/abs/1807.10108)
37. Zou L, Sun J, Gao M, Wan W, Gupta BB (2019) A novel coverless information hiding method based on the average pixel value of the sub-images. *Multimed Tools Appl* 78:7965–7980 (**Springer**)
38. Meng L, Jiang X, Zhang Z, Li Z, Sun T (2022) A robust coverless image steganography based on an end-to-end hash generation model. *IEEE Trans Circuits Syst Video Technol* 33(7):3542–3558

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Damaris Belle M. Fotso¹  · **Serge Alain Ebele**^{1,2,3}  · **Stéphane Gael R. Ekodeck**^{1,2,3}  · **René Ndoundam**^{1,2,3} 

✉ Stéphane Gael R. Ekodeck
ekodeckstephane@gmail.com

Damaris Belle M. Fotso
makwate.fotso@facsciences-uy1.cm

Serge Alain Ebele
sergeebele@gmail.com

René Ndoundam
ndoundam@gmail.com

¹ University of Yaounde I, LIRIMA, TEAM GRIMCAPE, Yaoundé, Cameroon

² IRD, UMI, UMMISCO, IRD France Nord, F-93143 Bondy, France

³ Sorbonne Universités, Univ. Paris 06, UMI 209, UMMISCO, F-75005 Paris, France